

PROJECT REPORT

Line Following Robot

Team 05 • Blade Runners

Department of Electronic & Telecommunication Engineering

12 November 2024

Team Members

E/22/151

E/22/215

E/22/405

E/22/058

E/22/017

E/22/219

E/22/022

Contents

1	Overview	1
2	Components Used	2
2.1	Robot Photographs	3
3	Design and Implementation	4
3.1	Chassis Design	4
3.1.1	Mechanical Layout	4
3.2	Circuit Design	5
3.2.1	Motor Driver Connections	5
3.2.2	Sensor Connections	6
4	Source Code	7
4.1	Full Arduino Sketch	7
4.2	Code Architecture Overview	9
4.2.1	Initialisation (<code>setup()</code>)	9
4.2.2	Sensor Decision Logic (<code>loop()</code>)	9
4.2.3	Motor Control (<code>rotateMotor()</code>)	10
5	Technical Specifications	11
6	Challenges and Solutions	12
6.1	1st Iteration	13
6.2	2nd Iteration	15
6.3	3rd Iteration	17
7	Performance Analysis	19
7.1	Iteration Comparison	19
7.2	Identified Limitations	19
8	Conclusion and Future Work	20
8.1	Conclusion	20
8.2	Future Work	20
	References	22

List of Figures

2.1	Physical build of the line-following robot	3
3.1	Circuit schematic – Arduino Uno, L298N motor driver, and IR sensor array	5
6.1	First iteration – initial hardware assembly	13
6.2	Second iteration – TSRT5000 sensor integration	15
6.3	Third iteration – final 3-sensor configuration	17

Chapter 1

Overview

Project Goal: Design and build a fully autonomous line-following robot using an Arduino-based control system, integrating hardware and software to achieve stable, real-time path tracking.

This project involved designing and building a line-following robot capable of detecting and tracking a black line on a white surface. The robot was constructed using essential components such as DC motors, an H-bridge motor driver, infrared sensors, and an Arduino Uno microcontroller.

The primary objectives of the project were:

- Understand and apply duty-cycle (PWM) control for precise motor speed regulation.
- Implement H-bridge (L298N) operation for bidirectional motor drive.
- Calibrate IR sensors and iterate on sensor selection and placement.
- Develop robust control logic capable of handling sharp turns and high-speed tracking.
- Gain hands-on experience in full-system integration: mechanical, electrical, and software.

The following students comprise **Team 05 – Blade Runners**:

Index No.	Role
E/22/151	Team Member
E/22/215	Team Member
E/22/405	Team Member
E/22/058	Team Member
E/22/017	Team Member
E/22/219	Team Member
E/22/022	Team Member

Chapter 2

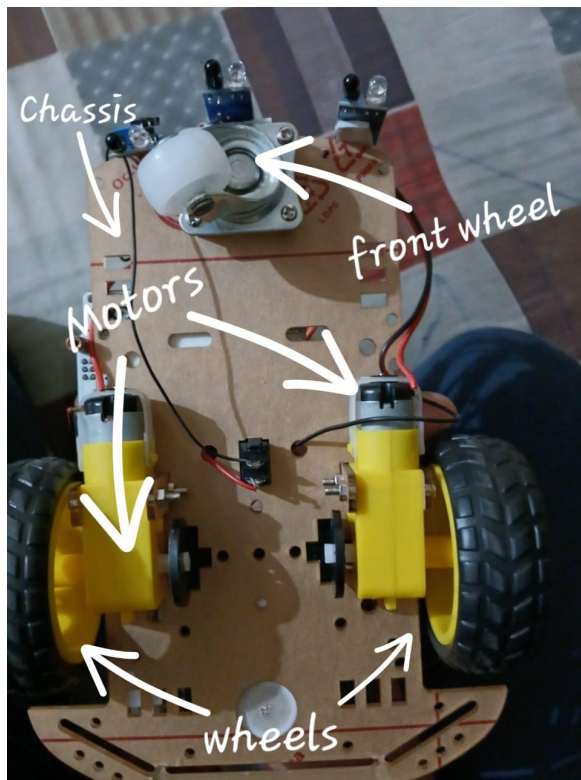
Components Used

The components below were selected to meet the functional and design requirements of the robot, evolving through three hardware iterations.

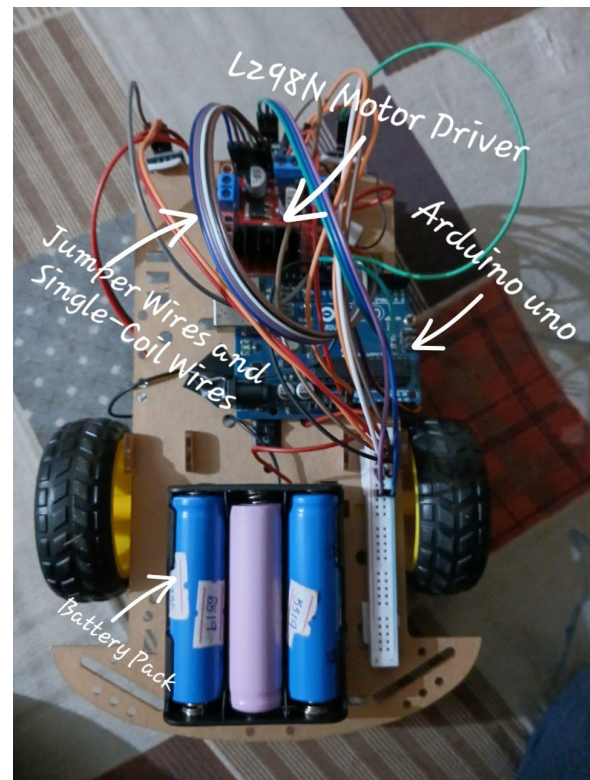
Table 2.1: Bill of Materials

Component	Model / Type	Purpose
Microcontroller	Arduino Uno (ATmega328P)	Central processing: reads sensors, drives motors
Motor Driver	L298N H-Bridge	Bidirectional speed & direction control of DC motors
Sensor (v1)	QTR-RC Array	Reflectivity detection – replaced due to ground-clearance issues
Sensor (v2)	TSRT5000 Analog	Improved clearance tolerance; 5-sensor array
Sensor (v3)	Individual Analog IR	Flexible placement; final configuration (3 sensors)
Motors	DC Geared (6–12 V)	Rear-wheel drive
Chassis	Wooden platform	Compact, lightweight mechanical base
Power Supply	3.7 V Li-ion $\times$ 3	Powers all components
Breadboard	Standard	Common-ground bus for analog sensors
Wiring	Jumper & coil wires	Interconnections
Soldering Iron	—	Permanent, stable electrical joints

2.1 Robot Photographs



(a) Chassis – front view



(b) Electronics assembly – top view

Figure 2.1: Physical build of the line-following robot

Chapter 3

Design and Implementation

3.1 Chassis Design

The robot's chassis is a compact, lightweight wooden platform engineered to balance weight distribution while housing all electronic components. The design decisions are summarised below.

3.1.1 Mechanical Layout

1. **Rear Drive Motors:** Two DC geared motors are mounted symmetrically on the rear axle, providing stable differential-drive traction for line-following manoeuvres.
2. **Front Caster Wheel:** A free-rotating caster wheel at the front allows smooth turning without binding, maintaining three-point contact stability.
3. **L298N Motor Driver:** Mounted centrally on top of the chassis, the L298N module controls both motor speed and direction via PWM and direction signals from the Arduino.
4. **Arduino Uno:** Acts as the robot's brain – processes IR sensor inputs and outputs motor control signals in real time.
5. **Battery Pack:** Three 3.7 V Li-ion cells positioned at the rear, balancing the chassis weight and providing sufficient current for both motors and logic.
6. **Wiring:** Jumper and single-coil wires connect all modules with minimal slack, organised to avoid mechanical interference with moving parts.
7. **Sensor Array:** Three IR sensors mounted at the front underside of the chassis, with spacing optimised through iterative testing.

3.2 Circuit Design

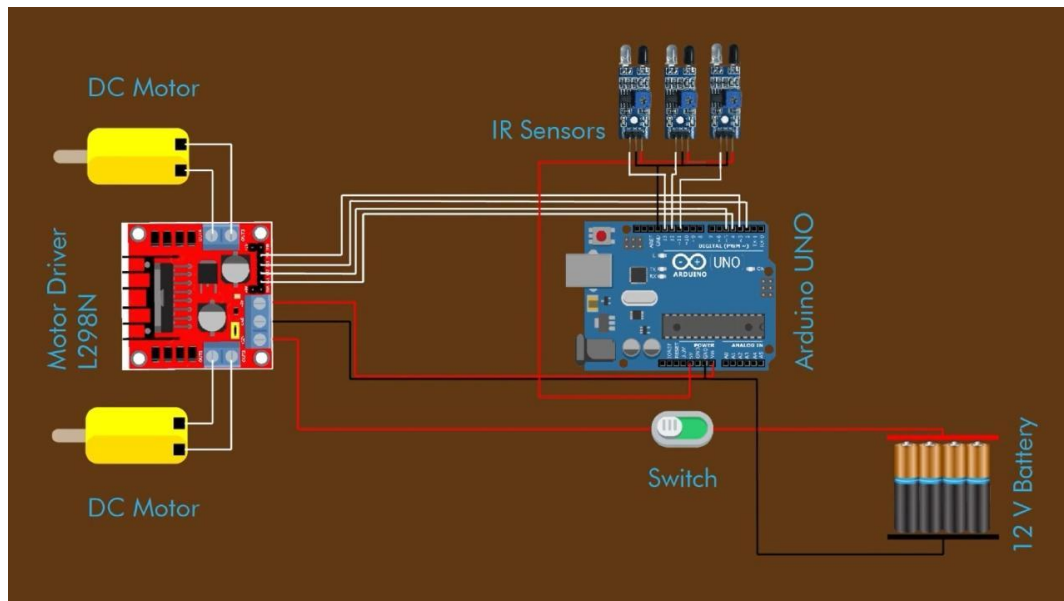


Figure 3.1: Circuit schematic – Arduino Uno, L298N motor driver, and IR sensor array

3.2.1 Motor Driver Connections

The L298N motor driver is interfaced with the Arduino as follows:

Table 3.1: L298N Pin Connections

L298N Pin	Connected To	Function
OUT1, OUT2	Right DC motor	Right motor drive
OUT3, OUT4	Left DC motor	Left motor drive
ENA	Arduino PWM pin 6	Right motor speed (duty cycle)
ENB	Arduino PWM pin 5	Left motor speed (duty cycle)
IN1, IN2	Arduino pins 7, 8	Right motor direction
IN3, IN4	Arduino pins 9, 10	Left motor direction

3.2.2 Sensor Connections

Table 3.2: IR Sensor Pin Connections

Sensor	Arduino Pin	Notes
Right IR sensor	Digital pin 11	INPUT mode
Left IR sensor	Digital pin 12	INPUT mode
Middle IR sensor	Digital pin 13	INPUT mode; added in 3rd iteration
Common ground	Breadboard GND bus	All sensor grounds unified

Chapter 4

Source Code

4.1 Full Arduino Sketch

```
1 // -- Pin Definitions
  -----
2 #define IR_SENSOR_RIGHT  11    // Right IR sensor
3 #define IR_SENSOR_LEFT   12    // Left IR sensor
4 #define IR_SENSOR_MIDDLE 13    // Centre IR sensor (added in 3rd
   iteration)
5 #define MOTOR_SPEED      185   // Base PWM speed (0-255)
6
7 // -- Motor Driver Pins
  -----
8 int enableRightMotor = 6;    // PWM pin -- right motor speed
9 int rightMotorPin1   = 7;    // Direction control
10 int rightMotorPin2   = 8;
11 int enableLeftMotor  = 5;    // PWM pin -- left motor speed
12 int leftMotorPin1    = 9;
13 int leftMotorPin2    = 10;
14
15 // -- Setup
  -----
16 void setup() {
17     // Override Timer0 prescaler for ~7812.5 Hz PWM on pins 5 & 6
18     // This reduces audible motor whine and improves control
       resolution
19     TCCR0B = TCCR0B & B11111000 | B00000010;
20
21     // Motor output pins
22     pinMode(enableRightMotor, OUTPUT);
23     pinMode(rightMotorPin1,   OUTPUT);
24     pinMode(rightMotorPin2,   OUTPUT);
25     pinMode(enableLeftMotor,  OUTPUT);
26     pinMode(leftMotorPin1,    OUTPUT);
27     pinMode(leftMotorPin2,    OUTPUT);
28
29     // Sensor input pins
30     pinMode(IR_SENSOR_RIGHT,  INPUT);
31     pinMode(IR_SENSOR_LEFT,   INPUT);
32     pinMode(IR_SENSOR_MIDDLE, INPUT);
33
34     rotateMotor(0, 0); // Ensure motors are stopped on power-up
```

```
35 }
36
37 // -- Main Loop
    -----
38 void loop() {
39     int R = digitalRead(IR_SENSOR_RIGHT);
40     int L = digitalRead(IR_SENSOR_LEFT);
41     int M = digitalRead(IR_SENSOR_MIDDLE);
42
43     /* Sensor logic:
44      * HIGH = sensor over BLACK line
45      * LOW  = sensor over WHITE surface
46      *
47      * Priority: centre > right > left > all-off > stop
48      */
49     if (M == HIGH) {
50         // Centre on line: go straight at full speed
51         rotateMotor(MOTOR_SPEED, MOTOR_SPEED);
52     }
53     else if (R == LOW && L == LOW) {
54         // All sensors off-line: slight right bias to recover
55         rotateMotor(MOTOR_SPEED + 10, MOTOR_SPEED);
56     }
57     else if (R == HIGH && L == LOW) {
58         // Drifted right: reverse right motor to steer right
59         rotateMotor(-MOTOR_SPEED * 0.6, MOTOR_SPEED);
60     }
61     else if (L == HIGH && R == LOW) {
62         // Drifted left: reverse left motor to steer left
63         rotateMotor(MOTOR_SPEED, -MOTOR_SPEED * 0.6);
64     }
65     else {
66         // Kill switch: both outer sensors on white -> halt
67         rotateMotor(0, 0);
68     }
69 }
70
71 // -- Motor Control Function
    -----
72 // rightMotorSpeed / leftMotorSpeed: positive = forward,
    // negative = reverse
73 void rotateMotor(int rightMotorSpeed, int leftMotorSpeed) {
74     // Right motor direction
75     if (rightMotorSpeed < 0) {
76         digitalWrite(rightMotorPin1, LOW);
77         digitalWrite(rightMotorPin2, HIGH);
78     } else if (rightMotorSpeed > 0) {
79         digitalWrite(rightMotorPin1, HIGH);
80         digitalWrite(rightMotorPin2, LOW);
81     } else {
```

```
82     digitalWrite(rightMotorPin1, LOW);
83     digitalWrite(rightMotorPin2, LOW);
84 }
85
86 // Left motor direction
87 if (leftMotorSpeed < 0) {
88     digitalWrite(leftMotorPin1, LOW);
89     digitalWrite(leftMotorPin2, HIGH);
90 } else if (leftMotorSpeed > 0) {
91     digitalWrite(leftMotorPin1, HIGH);
92     digitalWrite(leftMotorPin2, LOW);
93 } else {
94     digitalWrite(leftMotorPin1, LOW);
95     digitalWrite(leftMotorPin2, LOW);
96 }
97
98 // Set speed via PWM
99 analogWrite(enableRightMotor, abs(rightMotorSpeed));
100 analogWrite(enableLeftMotor, abs(leftMotorSpeed));
101 }
```

Listing 4.1: Line-following robot – Arduino sketch (final version)

4.2 Code Architecture Overview

4.2.1 Initialisation (setup())

- **PWM Frequency Override:** The line `TCCR0B = TCCR0B & B11111000 | B00000010` modifies Timer0's prescaler to run at ≈ 7812.5 Hz instead of the default 976 Hz. This significantly reduces audible motor whine and provides finer duty-cycle resolution.
- **Pin Modes:** All six motor-driver pins are set as OUTPUT; all three IR sensor pins are set as INPUT.
- **Safe Start:** `rotateMotor(0,0)` is called to prevent the robot from driving immediately on power-up.

4.2.2 Sensor Decision Logic (loop())

The control logic implements a simple finite-state machine based on the three binary sensor readings:

Table 4.1: Sensor State Truth Table

Middle (M)	Right (R)	Left (L)	Action
HIGH	any	any	Go straight (MOTOR_SPEED, MOTOR_SPEED)
LOW	LOW	LOW	Slight right correction (+10 to right motor)
LOW	HIGH	LOW	Turn right (right motor reversed at 60%)
LOW	LOW	HIGH	Turn left (left motor reversed at 60%)
LOW	HIGH	HIGH	Kill switch – stop both motors

4.2.3 Motor Control (rotateMotor())

The function accepts signed speed values and drives both motor direction and speed:

$$\text{PWM duty cycle} = \frac{|\text{speed}|}{255} \times 100\% \quad (4.1)$$

Direction is set by the IN1/IN2 (and IN3/IN4) pin pairs on the L298N. A negative speed value reverses the logic levels to reverse the motor, enabling in-place turning manoeuvres.

Chapter 5

Technical Specifications

Table 5.1: Component Technical Specifications

Component	Parameter	Value / Detail
	Microcontroller	ATmega328P
	Operating Voltage	5 V
	Digital PWM Pins	6 (pins 3, 5, 6, 9, 10, 11)
	Operating Voltage	5 V – 12 V
	Max Current per Channel	2 A
	Speed Control	ENA / ENB via PWM
	Logic Supply	5 V from Arduino
QTR-RC Sensor	Detection Range	1 mm – 3 mm
	Output	Digital / Analog reflectivity
TSRT5000 Sensor	Sensors	5-element analog array
	Advantage	Greater ground-clearance tolerance
DC Geared Motors	Voltage	6 V – 12 V
	Type	Geared (high-torque)
Battery Pack	Cell Type	18650 Li-ion (3.7 V each)
	Configuration	3 cells in series $\approx$ 11 V

Chapter 6

Challenges and Solutions

The robot was developed through three hardware/software iterations. Each iteration resolved specific problems uncovered during testing.

6.1 1st Iteration

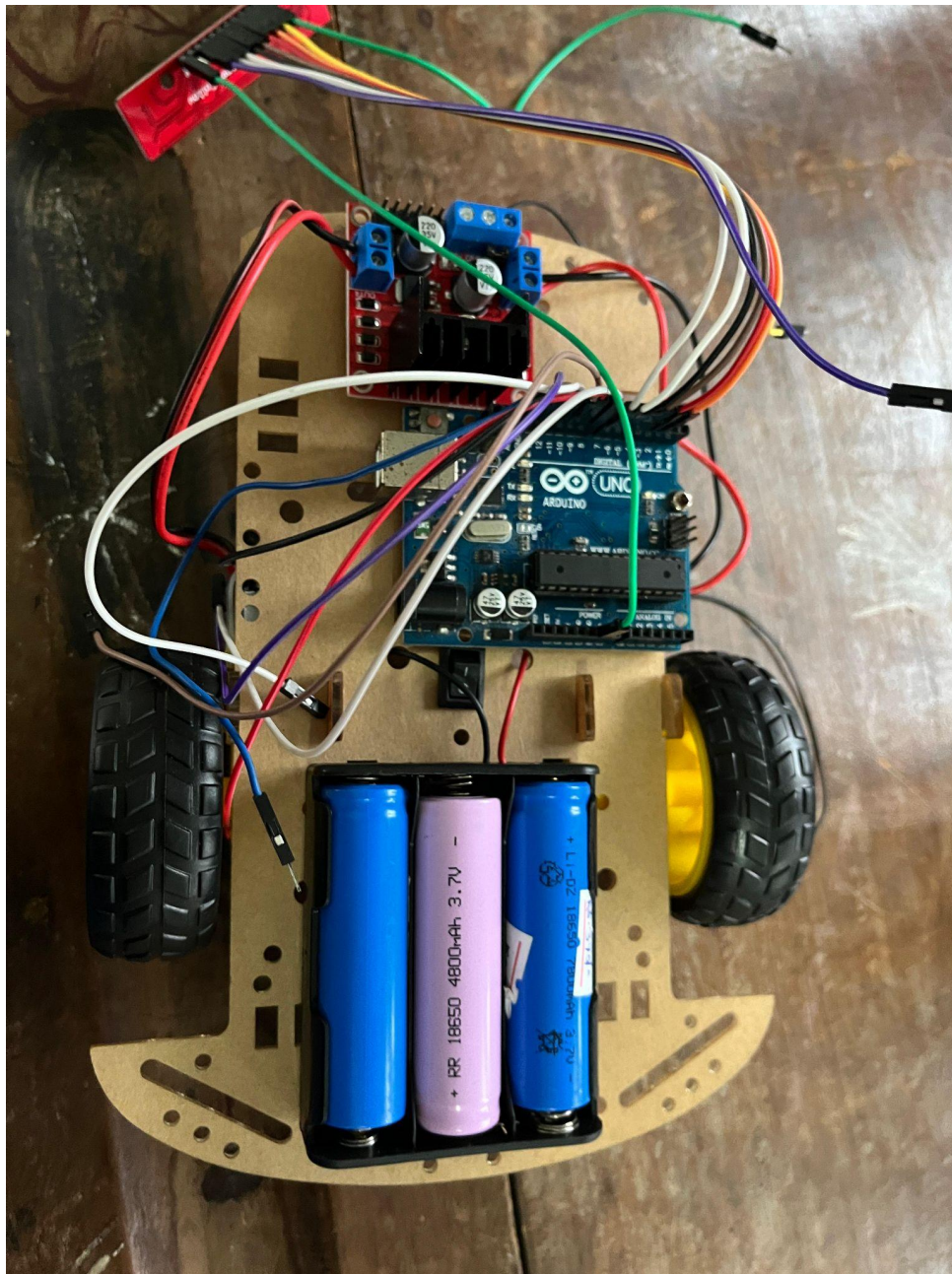


Figure 6.1: First iteration – initial hardware assembly

Problem 1: Speed Control Failure

Issue: Motors ran at a fixed speed with no PWM control.

Root Cause: ENA and ENB pins were wired to digital (non-PWM) Arduino pins.

Solution: Reconnected ENA to pin 6 and ENB to pin 5 – both PWM-capable pins. This enabled effective duty-cycle control via `analogWrite()`.

Problem 2: Sensor Calibration Instability

Issue: QTR sensor produced inconsistent line readings.

Root Cause: Digital pin connections limited output resolution to a binary signal, discarding the sensor's analog resolution.

Solution: Rewired the QTR sensor to analog pins, restoring full-range reflectivity readings and significantly improving detection consistency.

Problem 3: Ground Clearance Mismatch

Issue: QTR sensor required ≤ 3 mm ground proximity, but chassis clearance exceeded this.

Root Cause: Chassis design did not account for the sensor's tight proximity requirement.

Solution: Replaced the QTR array with a TSRT5000 analog sensor, which tolerates greater ground clearance and provides stable readings at typical chassis heights.

6.2 2nd Iteration

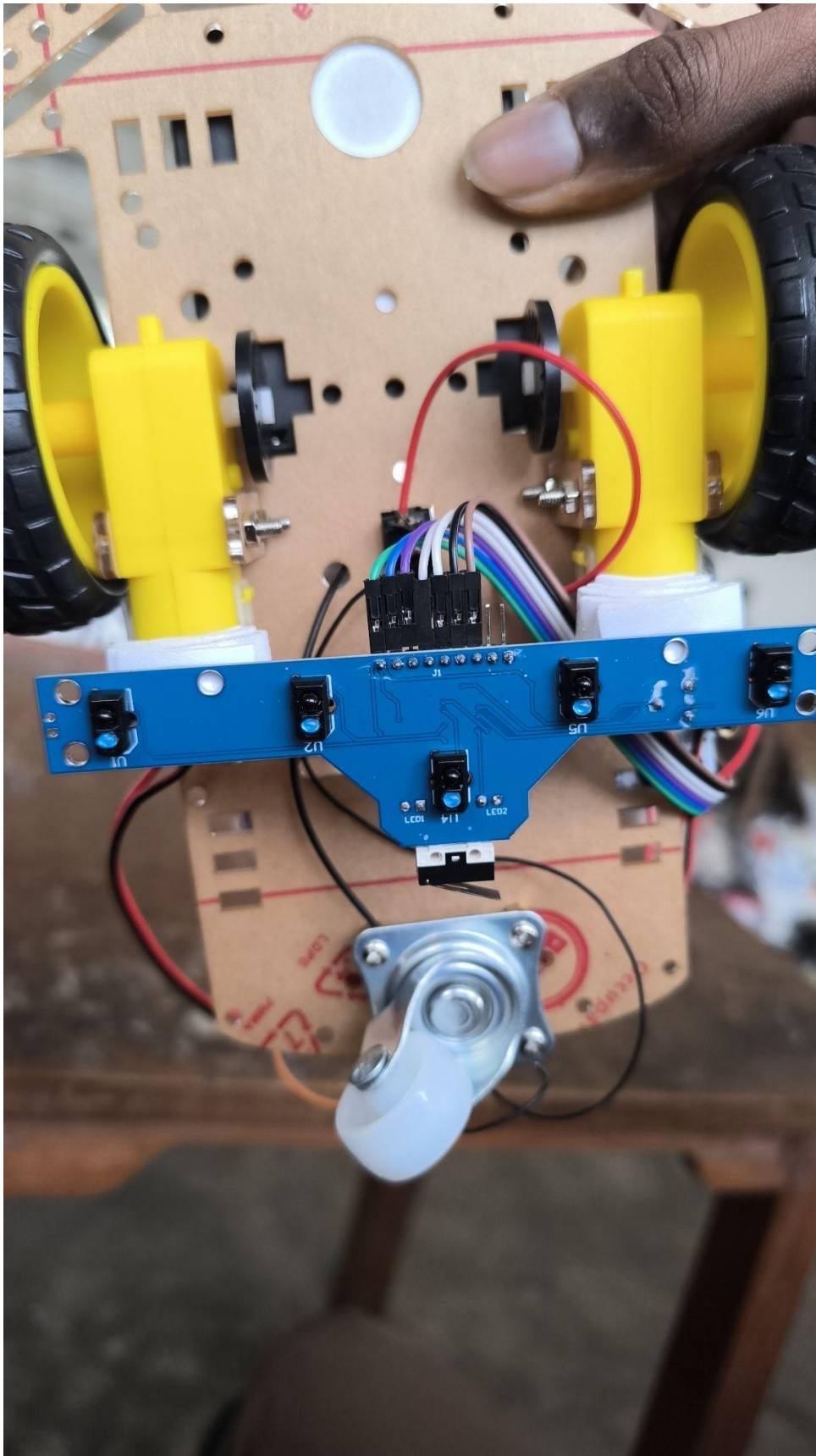


Figure 6.2: Second iteration – TSRT5000 sensor integration

Problem 4: Sensor Placement

Issue: The TSRT5000 sensor was too large to mount at the front with appropriate clearance.

Root Cause: Insufficient front-chassis space; rear mounting would require inverting all control logic.

Solution: Mounted the sensor beneath the chassis centre using multiple layers of double-sided tape, achieving the required height within the available geometry.

Problem 5: Lack of a Kill Switch

Issue: The robot would veer off course during testing and could only be stopped manually.

Root Cause: No software-defined stop condition for out-of-bounds scenarios.

Solution: Added a kill-switch condition: when all sensors simultaneously detect white (binary HIGH, HIGH, HIGH), both motors are halted. This effectively stops the robot at the end of the track or when it loses the line entirely.

Problem 6: Erratic / Oscillatory Movement

Issue: Robot oscillated severely; only stable at very low speeds.

Root Cause: The TSRT5000 array has sensors in a staggered front-to-back configuration, complicating the positional logic and causing high-frequency corrections.

Solution 1: Replaced PID-based code with a simple state-machine that considers all possible 5-sensor binary combinations, eliminating incorrect high-gain corrections.

Solution 2: Replaced the TSRT5000 with three individual analog IR sensors arranged in a side-by-side (horizontal) configuration, giving explicit left/centre/right positional feedback with no stagger ambiguity.

6.3 3rd Iteration

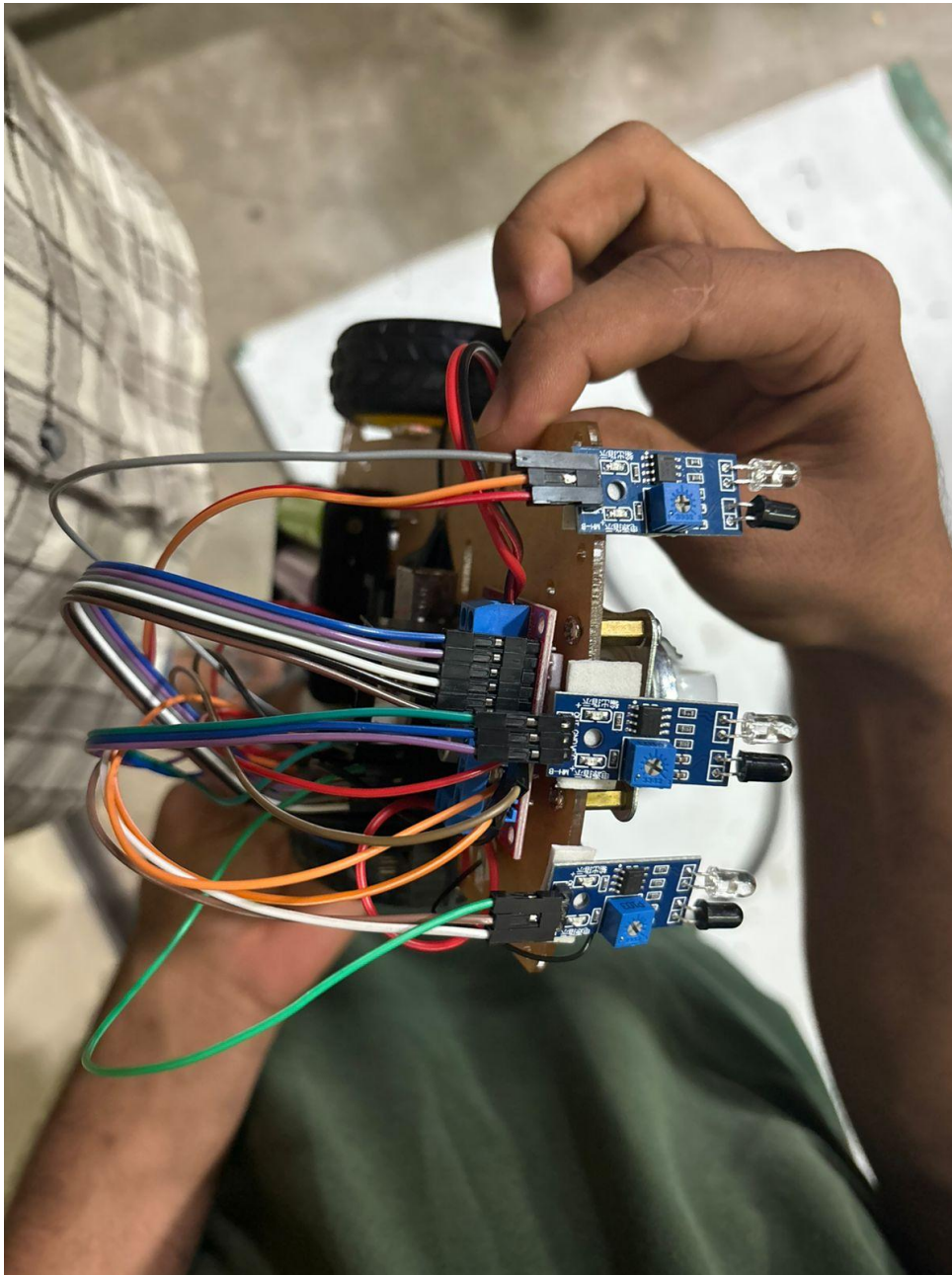


Figure 6.3: Third iteration – final 3-sensor configuration

Problem 7: Line Detection at High Speed

Issue: Robot missed the line at higher motor speeds with only two sensors.

Root Cause: Binary left/right feedback provides insufficient spatial resolution; any lateral offset causes the robot to lose the line before correcting.

Solution: Added a centre sensor (pin 13). When the centre sensor detects the line, both motors run at equal speed, providing stable straight-line tracking without triggering a corrective turn.

Problem 8: Sharp Turn Handling

Issue: Robot loses the line during sharp curves.

Root Cause: Fixed sensor spacing provides limited lateral coverage; narrow arcs exit the detection window before a correction is applied.

Solution: Increased the spacing between the outer sensors to extend lateral coverage. The corrective turn speed was also tuned (60% of `MOTOR_SPEED` in reverse) to produce a tighter turning radius without overshoot.

Problem 9: Centre Sensor Inconsistency

Issue: Centre sensor intermittently failed to detect the line.

Root Cause: If the physical line width is narrower than expected, or sensor height varies, the centre sensor can sit between two detectable edges.

Solution: Adjusted sensor vertical height and horizontal position based on measured line width. A threshold approach was also implemented: when the centre sensor loses the line, the two outer sensors take priority for directional correction.

Chapter 7

Performance Analysis

7.1 Iteration Comparison

Table 7.1: Performance Comparison Across Iterations

Metric	Iter 1	Iter 2	Iter 3	Notes
Sensor Type	QTR-RC	TSRT5000	3× Analog IR	Final config
Speed Control	Fixed	PWM	PWM	Fixed in Iter 1
Line Stability	Poor	Moderate	Good	Oscillation re-solved
Sharp Turns	Fail	Partial	Good	Centre sensor key
Kill Switch	No	Yes	Yes	Added Iter 2
High-Speed Track	No	No	Yes	3-sensor logic

7.2 Identified Limitations

Despite the improvements achieved, the following limitations were identified:

1. **No PID Controller:** The current non-PID logic produces step corrections (full reverse at 60%) rather than proportional corrections. A PID controller would smooth the trajectory, especially at higher speeds.
2. **Lighting Sensitivity:** IR sensors are susceptible to ambient light variation. Bright or uneven lighting on the track surface caused occasional misreadings.
3. **Power Stability:** Battery voltage sag at high motor load occasionally caused sensor and logic supply dips, leading to transient erratic behaviour.
4. **Track Complexity:** The robot performs well on standard oval/rectangular tracks but struggles with very sharp 90° corners and intersections.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

This project successfully designed, built, and iterated on an autonomous line-following robot using an Arduino Uno microcontroller. Through three hardware iterations the team:

- Resolved sensor instability by migrating from a QTR array to individual analog IR sensors.
- Achieved PWM-based speed control by correctly wiring ENA/ENB to Timer0 PWM pins.
- Implemented a software kill switch for safe operation during testing.
- Added a centre sensor to reduce oscillation and improve high-speed line tracking.
- Tuned corrective turn speeds to handle moderate curves reliably.

The final robot achieved stable line-following at moderate speeds with reliable turn handling, meeting the project’s core objectives and providing the team with valuable hands-on experience in embedded systems, control logic, and iterative engineering design.

8.2 Future Work

1. **PID Controller:** Implement a Proportional-Integral-Derivative control loop. The error signal would be the weighted sum of sensor positions, enabling smooth, proportional corrections and significantly reducing oscillation at high speeds.

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (8.1)$$

2. **Advanced Sensors:** Replace analog IR sensors with a dedicated line-sensor array (e.g. Pololu QTR-8RC) offering higher spatial resolution and built-in calibration routines.
3. **Regulated Power Supply:** Add a 5V buck converter to decouple logic and sensor supply from the motor power rail, eliminating voltage-sag issues.
4. **Obstacle Avoidance:** Integrate an HC-SR04 ultrasonic sensor for forward-obstacle detection, enabling the robot to pause and resume tracking around obstacles.

5. **Bluetooth Telemetry:** Add an HC-05 Bluetooth module to stream sensor readings and motor states to a mobile app for real-time debugging without a physical connection.

References

- [1] Last Minute Engineers. (n.d.). *In-Depth: Interface L298N DC Motor Driver Module with Arduino*. Available at: <https://lastminuteengineers.com/l298n-dc-stepper-driver-arduino/> [Accessed 05 Nov 2024].
- [2] NI. (n.d.). *PID Theory Explained*. Available at: <https://www.ni.com/en/shop/labview/pid-theory-explained.html> [Accessed 05 Nov 2024].
- [3] Mechatronics, R. (2023). *Arduino based Line Following Robot Tutorial*. YouTube, 21 May. Available at: <https://youtu.be/w3bE3ygJMx8> [Accessed 05 Nov 2024].
- [4] Electronics Hub. (2022). *How to Make an Arduino Based Line Follower Robot?* Available at: <https://www.electronicshub.org/arduino-line-follower-robot/> [Accessed 07 Nov 2024].
- [5] Circuit Digest. (2022). *Arduino Line Follower Robot Project Using Motor Driver and IR Sensor*. Available at: <https://circuitdigest.com/microcontroller-projects/arduino-uno-line-follower-robot> [Accessed 07 Nov 2024].
- [6] Quartz Components. (n.d.). *Line Follower Robot using Arduino*. Available at: <https://quartzcomponents.com/blogs/electronics-projects/line-follower-robot-using-arduino/> [Accessed 08 Nov 2024].